

Clase 4: Orquestación de los datos

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Contenido

- 1 Introducción a la Orquestación de Datos
 - ¿Qué es la Orquestación?
 - Apache Airflow
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Contenido

- 1 Introducción a la Orquestación de Datos
 - ¿Qué es la Orquestación?
 - Apache Airflow
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

¿Qué es la Orquestación?

Definición

- La **orquestación**, en el contexto de la ingeniería de datos, es como el **director de una orquesta**.
- Su función es asegurar que cada **instrumento** (cada tarea, como la limpieza de datos o el entrenamiento de un modelo) comience y termine en el momento preciso y de la forma esperada.

¿Qué es la Orquestación?

Importancia en la Ingeniería de Datos

- Sin orquestación, un pipeline complejo requeriría:
 - Ejecución manual de los pipelines, proceso propenso a errores.
 - Monitoreo manual constante para saber cuándo terminó una tarea y cuándo debe comenzar la siguiente.
 - Identificación manual de pipelines fallidos y posterior alerta.

¿Qué es la Orquestación?

Importancia en la Ingeniería de Datos

- **Fiabilidad y Consistencia:** Los datos se procesan siempre en el mismo orden, minimizando errores y asegurando que las predicciones de ML se basen en datos limpios y actualizados.
- **Escalabilidad:** Permite que los flujos de trabajo se adapten al crecimiento del volumen de datos (ej. más carreras, más helicópteros) al gestionar la ejecución distribuida de tareas.
- **Eficiencia de Recursos:** Solo se ejecutan los recursos de cómputo cuando son estrictamente necesarios, optimizando los costos.

El Problema de los Cron Jobs

¿Por qué la programación tradicional falla en Big Data?

Cuando los pipelines crecen, los scripts programados en **cron** se vuelven insostenibles:

- **Falta de dependencias:** Si la Tarea A (Ingesta) se retrasa, la Tarea B (Transformación) se ejecuta igual sobre datos incompletos o corruptos.
- **Caja Negra:** No hay visibilidad centralizada del estado de las tareas (¿corrió?, ¿falló?, ¿cuánto tardó?).
- **Manejo de Errores Inexistente:** Si un script falla, no hay reintentos automáticos ni alertas nativas.
- **Idempotencia Comprometida:** Reprocesar un día específico del pasado de forma manual es un proceso propenso a errores humanos.

Contenido

- 1 Introducción a la Orquestación de Datos
 - ¿Qué es la Orquestación?
 - Apache Airflow
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Apache Airflow

¿Qué es?

- Es una plataforma de código abierto, originada en Airbnb.
- Permite crear, programar, automatizar y monitorear/supervisar flujos de trabajo complejos (data pipelines) de manera programática usando Python.

¿Qué es Apache Airflow?

De Scripts Aislados a Grafos Orquestados

- **Workflows como Código (Configuration as Code):** Los pipelines se definen completamente en Python. Si se puede programar en Python, Airflow lo puede orquestar.
- **Arquitectura Distribuida:** Capaz de escalar desde un entorno de desarrollo local hasta clusters empresariales de miles de tareas diarias.
- **Extensible:** Se integra de forma nativa con todo el ecosistema de datos (Apache Beam, dbt, PostgreSQL, Spark, AWS, GCP, etc.).

Apache Airflow

¿Qué es un flujo de trabajo en Airflow?

- Un flujo de trabajo en Airflow se representa como un DAG (Directed Acyclic Graph) o Grafo Dirigido Acíclico.
- Los DAGs son colecciones de tareas (tasks) organizadas con dependencias y que se ejecutan en un orden lógico específico, sin formar ciclos.
- Permite definir la secuencia de ejecución, las dependencias entre tareas y los reintentos en caso de fallo.

Apache Airflow

¿Para qué se usa Apache Airflow?

- Orquestación de flujos de datos.
- Automatización de tareas.
- Gestión de ETL y ML.

Apache Airflow

Características principales

- **Código como definición:** Los flujos de trabajo se definen en scripts de Python, lo que ofrece flexibilidad y permite versionar los workflows.
- **Interfaz de usuario:** Dispone de una interfaz web para monitorizar el estado de las ejecuciones, visualizar los DAGs y gestionar las tareas.
- **Extensibilidad:** Incluye una gran cantidad de operadores que facilitan la integración con servicios de AWS, Azure, GCP y otras tecnologías externas.
- **Escalabilidad:** La plataforma está diseñada para manejar desde un pequeño número de tareas hasta miles de ellas de forma diaria.

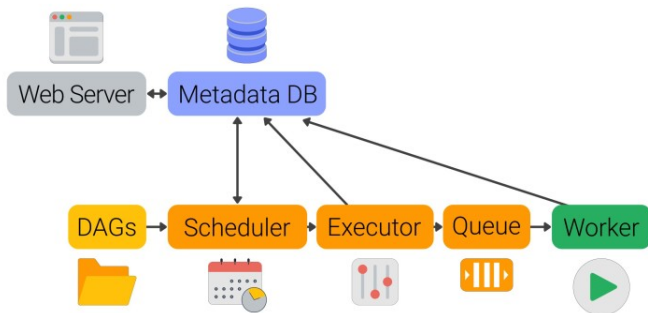
Apache Airflow

Problemas que Resuelve

- **Gestión de Dependencias:** **A** debe terminar antes de que **B** comience.
- **Programación (Scheduling):** ejecución en base a reglas de tiempo o eventos.
- **Monitoreo y Logging:** interfaz gráfica centralizada para supervisar el estado de cada tarea.
- **Manejo de Errores y Reintentos:**
 - Configuración de notificaciones cuando falla una tarea.
 - Configuración de reintentos hasta X veces antes de marcar la tarea fallida.

Apache Airflow

Componentes



¿Dónde se Ejecuta Airflow?

Entornos de Arquitectura

- **Local / Docker (Desarrollo):** Levantado en una máquina local usando contenedores independientes para el Webserver, Scheduler y la Base de Datos de Metadatos (PostgreSQL). Herramientas como Astro CLI facilitan este proceso.
- **Kubernetes (Escalabilidad):** Uso del KubernetesExecutor para levantar pods dinámicos por cada tarea, optimizando los recursos del cluster.
- **Servicios Gestionados (Cloud de Producción):**
 - **GCP:** Cloud Composer.
 - **AWS:** Managed Workflows for Apache Airflow (MWAA).

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
 - DAGs (Directed Acyclic Graphs)
 - Tasks (Tareas)
 - Operator (Operadores)
 - Tiempo
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
 - DAGs (Directed Acyclic Graphs)
 - Tasks (Tareas)
 - Operator (Operadores)
 - Tiempo
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

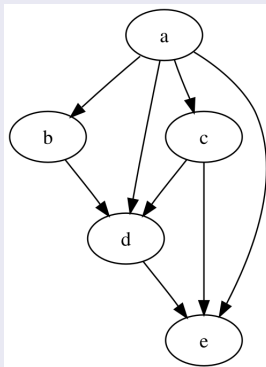
Directed Acyclic Graph (Grafo Acíclicos Dirigido)

- **Directed (Dirigido):** Las tareas tienen un orden de ejecución explícito y un sentido lógico único (flujo aguas abajo o downstream).
- **Acyclic (Acíclico):** El grafo no permite bucles infinitos. La Tarea A no puede depender de la Tarea B si la Tarea B ya depende de la Tarea A.
- **Graph (Grafo):** Estructura matemática compuesta por vértices (Tareas) y aristas (Dependencias).

DAGs (Directed Acyclic Graphs)

DAG

- Grafo Acíclico Dirigido o Directed Acyclic Graph



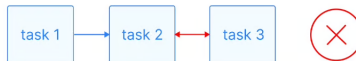
DAGs (Directed Acyclic Graphs)

DAG

Directed

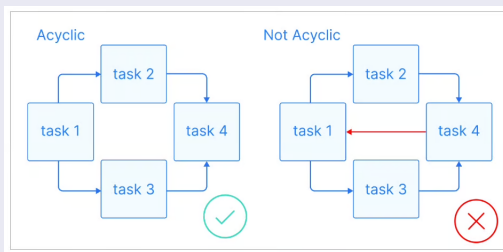


Not directed



DAGs (Directed Acyclic Graphs)

DAG



Airflow DAG

En Airflow, un flujo de trabajo se define como un DAG:

- Es la representación fundamental de un flujo de trabajo en Airflow.
- Es una colección de todas las tareas que desea ejecutar, organizada de manera que muestre sus relaciones y dependencias.

Airflow DAG

Estructura y Crucialidad

- El DAG es el corazón de Airflow porque no contiene la lógica real de lo que hace el código (eso reside en las tareas), sino que define la arquitectura de las operaciones:
 - **Nodos (Tasks):** Representan unidades discretas de trabajo (ej., la limpieza de datos, la ejecución de una consulta SQL, el entrenamiento de un modelo ML).
 - **Aristas (Dependencies):** Son las flechas que conectan los nodos. Definen las reglas de ejecución: *Esta tarea solo puede comenzar si la tarea anterior ha finalizado con éxito.*
- Un DAG podría ser el flujo completo: Limpiar Datos — > Extraer Características — > Alimentar el Modelo ML.

Estructura de un DAG en Código

Anatomía de un Script de Airflow

Un archivo de DAG estándar en Python se compone de 4 secciones obligatorias:

- **Imports:** Módulos de Airflow y dependencias externas.
- **Default Arguments:** Diccionario con configuraciones de resiliencia (dueño, reintentos, políticas de fallo).
- **Instanciación del DAG:** Definición del contexto, ID, frecuencia de ejecución (schedule) y fechas de control.
- **Declaración de Tareas y Dependencias:** Instanciación de operadores y definición del orden del flujo usando operadores binarios (`>>`, `<<`).

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 **Conceptos Fundamentales de Airflow**
 - DAGs (Directed Acyclic Graphs)
 - **Tasks (Tareas)**
 - Operator (Operadores)
 - Tiempo
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Tasks (Tareas)

Definición

- Una Tarea es la unidad de trabajo atómica y discreta dentro de un DAG.
- Representa un solo paso lógico en su flujo de datos.
- Las tareas en Airflow se definen como instancias de un Operador.
- Cada tarea tiene un identificador único (`task_id`), puede tener un número definido de reintentos (`retries`), y un tiempo máximo de ejecución (`timeout`).
- La granularidad de las tareas es importante; deben ser lo suficientemente pequeñas para que, si fallan, se puedan reintentar sin rehacer mucho trabajo.

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
 - DAGs (Directed Acyclic Graphs)
 - Tasks (Tareas)
 - **Operator (Operadores)**
 - Tiempo
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Operator (Operadores)

Definición

- Son plantillas predefinidas que encapsulan la lógica de ejecución para un tipo de trabajo específico.
- Son la capa que permite a los ingenieros definir qué debe hacer una tarea sin preocuparse de cómo debe ejecutarse en el entorno de Airflow.
- **Función:** Simplifican la creación de tareas, ya que se limitan a recibir parámetros y ejecutar la lógica necesaria (ej., conectarse a una base de datos o ejecutar un comando de sistema).

Tareas vs Operadores

Los bloques de construcción del Pipeline

Las tareas dentro de un DAG se ejecutan mediante Operadores. Un operador es la plantilla que define qué acción se va a realizar.

- **BashOperator:** Permite ejecutar comandos de consola (Shell scripts, comandos CLI). Es la herramienta estándar para disparar ejecuciones locales o llamadas de sistema.
- **PythonOperator:** Ejecuta funciones nativas de Python. Permite manipular datos en memoria, invocar APIs o ejecutar scripts personalizados.
- **Sensors (Sensores):** Operadores especiales de tipo abstracto que pausan el pipeline hasta que se cumpla una condición externa (ej: FileSensor espera a que aparezca un archivo en un directorio).

Operadores Core

Otros Operadores Comunes para Orquestación (ETL/ML)

- PythonOperator: Ejecuta cualquier función de Python. Ideal para lógica personalizada.
- BashOperator: Ejecuta comandos de shell (Bash, script de sistema).
- SQLExecuteOperator: Ejecuta comandos SQL en una base de datos configurada.
- KubernetesPodOperator: Lanza un pod de Kubernetes para ejecutar una tarea.
- ExternalTaskSensor: Espera a que una tarea en otro DAG se complete.

Operadores

Casos de uso

- PythonOperator: Ejecutar el script de Extracción de Características o el código que realiza el preprocesamiento de los datos de telemetría.
- BashOperator: Ejecutar comandos para activar un cluster de Spark, iniciar una transferencia de archivos S3, o limpiar directorios de logs.
- SQLExecuteOperator: Ejecutar un INSERT o un UPDATE en la Tabla de Posiciones después de que los datos de la vuelta se hayan limpiado y calculado.

Operadores

Casos de uso

- **KubernetesPodOperator:** Usado a menudo para entrenar modelos ML con alta demanda de recursos, encapsulando el entorno (imágenes de Docker) y el código.
- **ExternalTaskSensor:** Crucial para la orquestación multitable; el pipeline de Predicciones espera hasta que el pipeline de Limpieza de la Tabla de Rendimiento haya finalizado.

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 **Conceptos Fundamentales de Airflow**
 - DAGs (Directed Acyclic Graphs)
 - Tasks (Tareas)
 - Operator (Operadores)
 - **Tiempo**
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Conceptos Críticos de Tiempo

Scheduling, Catchup y Backfilling

- **start_date:** La fecha de inicio histórica del DAG. No es el momento en que se activa el interruptor en la UI, sino la fecha a partir de la cual Airflow calcula los intervalos.
- **schedule** (Planificación): Expresión Cron o timedelta que dicta la frecuencia del pipeline (ej: @daily, 0 5 * * *).
- **catchup:** Si se configura en True, Airflow buscará y ejecutará automáticamente todas las ejecuciones pasadas que no se hayan corrido desde la start_date hasta el día de hoy.
- **backfilling:** Comando CLI que permite al ingeniero forzar manualmente la ejecución del DAG sobre un rango de fechas históricas específicas tras un cambio de lógica.

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
 - DAGs (Directed Acyclic Graphs)
 - Tasks (Tareas)
 - Operator (Operadores)
 - Tiempo
 - Creación del primer DAG
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio

Ejemplo práctico

- https://github.com/ctorresf/developer-guides/blob/main/apache_airflow/vscode-devcontainer-airflow3-examples/src/dags/basic_dag.py
- https://github.com/ctorresf/developer-guides/blob/main/apache_airflow/vscode-devcontainer-airflow3-examples/src/dags/simple_hello_world.py
- https://github.com/ctorresf/developer-guides/blob/main/apache_airflow/vscode-devcontainer-airflow3-examples/src/dags/data_transfer_xcom.py

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo**
 - Interfaz gráfica
 - Buenas prácticas
 - Ejemplo ETL
- 4 Caso de Estudio

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo**
 - Interfaz gráfica**
 - Buenas prácticas
 - Ejemplo ETL
- 4 Caso de Estudio

Interfaz gráfica

Documentación oficial

- <https://airflow.apache.org/docs/apache-airflow/2.11.0/ui.html>
- <https://airflow.apache.org/docs/apache-airflow/stable/ui.html>

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo**
 - Interfaz gráfica
 - Buenas prácticas**
 - Ejemplo ETL
- 4 Caso de Estudio

Buenas Prácticas para DAGs

Parametrización de DAGs

- Parametros de entrada:
<https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/params.html>
- Airflow Variables:
<https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/variables.html>
- Jinja Templates: <https://airflow.apache.org/docs/apache-airflow/stable/templates-ref.html>

Buenas Prácticas para DAGs

Organización del Código

- Separación de la Lógica: el archivo DAG solo contiene la definición del DAG, las tareas, y las dependencias (la arquitectura). La lógica de negocios (Código de las transformaciones) debe estar en archivos separados.
- Conexión a recursos externos:
<https://airflow.apache.org/docs/apache-airflow/stable/authoring-and-scheduling/connections.html>

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo**
 - Interfaz gráfica
 - Buenas prácticas
 - Ejemplo ETL**
- 4 Caso de Estudio

Ejemplo ETL

- https://github.com/ctorresf/developer-guides/blob/main/apache_airflow/vscode-devcontainer-airflow3-examples/src/dags/complex_etl_with_beam.py

Contenido

- 1 Introducción a la Orquestación de Datos
- 2 Conceptos Fundamentales de Airflow
- 3 Monitoreo y Prácticas de Desarrollo
- 4 Caso de Estudio**

Caso de Estudio

Sistema de Consolidación Global-Mart

Objetivo General:

- Automatizar, bajo un esquema resiliente y con dependencias explícitas, el flujo completo de ingeniería de datos desarrollado en las clases anteriores.

Caso de Estudio

Sistema de Consolidación Global-Mart

Objetivos Específicos:

- Ejecutar el pipeline de Apache Beam para limpiar las ventas y los logs de estados de las sucursales, generando la Capa Silver (Parquet).
- Detectar automáticamente la existencia del archivo Parquet.
- Disparar el modelado analítico en dbt para construir los KPIs de la Capa Gold.
- Configurar un mecanismo de alerta inmediata que capture cualquier excepción del flujo.

Solución Paso a Paso del DAG

Diseño del Flujo Lógico en Código (1/2)

Para cumplir con el caso de estudio, implementaremos un DAG estructurado en 4 pasos:

- **Paso 1:** Configuración de Alertas (on_failure_callback): Definición de una función de Python que se dispare automáticamente si una tarea del grafo falla.
- **Paso 2:** Tarea de Ingesta (Beam): Un BashOperator que invoca el script de Apache Beam, procesando los CSVs y escribiendo el Parquet anidado.

Solución Paso a Paso del DAG

Diseño del Flujo Lógico en Código (2/2)

- **Paso 3:** Validación de Datos (Sensor): Un FileSensor que monitorea el Data Lake local, bloqueando el avance hasta confirmar la existencia del archivo .parquet.
- **Paso 4:** Tarea de Transformación (dbt): Un BashOperator final que ejecuta los comandos dbt run y dbt test para generar el reporte de finanzas en PostgreSQL.

Implementación de la Solución (Código)

- https://github.com/ctorresf/developer-guides/tree/main/apache_airflow/vscode-devcontainer-airflow3-docker-compose-etl

Conclusiones de la Clase

Buenas Prácticas de Orquestación

- **Tareas Atómicas e Idempotentes:** Cada tarea del DAG debe hacer una sola cosa. Si la tarea se ejecuta dos veces con los mismos parámetros, el resultado final debe ser idéntico sin duplicar datos.
- **Los DAGs no procesan datos:** Airflow es un director de orquesta, no los músicos. La computación pesada debe ocurrir dentro de Beam, Spark o dbt; Airflow solo da la orden de inicio.
- **Manejar siempre los límites de tiempo (Timeouts):** Los sensores deben tener siempre un parámetro timeout configurado para evitar que una tarea se quede colgada indefinidamente consumiendo recursos del sistema.

Clase 4: Orquestación de los datos

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026